

Project Evidentia

Target Root Path: `/home/kcplibrary/Documents/evidentia`

This document outlines the step-by-step procedure to initialize and configure the development environment for **Project Evidentia**, leveraging a modern stack consisting of a Vite + React + Tailwind CSS frontend and a PHP backend.

1. Directory Creation & Frontend Initialization

Navigate to your document storage directory, create the parent project container, and bootstrap the frontend using Vite:

```
# Navigate to projects directory
cd /home/kcplibrary/Documents

# Create main project directory and enter it
mkdir evidentia
cd evidentia

# Initialize the frontend boilerplate (Vite + React)
npm create vite@latest frontend -- --template react

# Navigate into frontend and install package dependencies
cd frontend
npm install
```

2. Tailwind CSS Integration

Execute the following commands within the `frontend/` directory to install Tailwind CSS along with its peer dependencies, and generate configuration files.

Step A: Install via npm

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step B: Configure Template Paths

Update the `tailwind.config.js` file to specify the entry points and component directories where Tailwind utilities will be utilized:

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './index.html',
    './src/**/*..{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

Step C: Inject Tailwind Directives

Replace or prepend the content of `src/index.css` with the standard Tailwind directives:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

3. Backend Setup (PHP)

Return to the root directory of the project to isolate and construct the PHP environment.

```
# Escape frontend to root directory (/home/kcplibrary/Documents/evidentia)
cd ..

# Create and enter the backend environment
mkdir backend
cd backend
```

Backend Strategy & Architectural Choice:

- **Option A (Microservice / API-driven):** Create lightweight script structures (e.g., `index.php`) directly. Serve natively using the built-in development engine:

```
php -S localhost:8000
```

- **Option B (Structured Package Management):** If external libraries, database ORMs, or third-party routing routers are expected, initialize Composer:

```
composer init
```

4. Final Architecture Matrix

Your directory topology should reflect the following structure upon successful deployment:

```
evidentia/  
├── backend/  
│   └── (PHP Core / API Scripts)  
├── frontend/  
├── node_modules/  
├── src/  
│   ├── App.jsx  
│   └── index.css  
├── index.html  
├── package.json  
├── tailwind.config.js  
└── vite.config.js
```

5. Concurrent Execution Workflow

To run the development environments in parallel, use two distinct terminal instances:

- **Frontend Terminal:** `cd frontend && npm run dev`
- **Backend Terminal:** `cd backend && php -S localhost:8000`

Critical Cross-Origin Resource Sharing (CORS) Notice

Because the Vite dev server (typically running on `http://localhost:5173`) and the PHP backend server occupy distinct network ports, ensure your entry PHP scripts explicitly declare headers allowing cross-origin requests to prevent asynchronous fetch blocking:

```
header("Access-Control-Allow-Origin: *");  
header("Access-Control-Allow-Headers: *");
```